```
# ================================================
# Roll no : 60       PRN: 0124UITM1060
# Name : Kshitij Shinde
# Dept. : Information Technology (Third Year)
# ================================================
# Assignment 3:
# Loan Approval Prediction using Logistic Regression
# ================================================

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

# ----------------------------
# Load Dataset
# ----------------------------
df = pd.read_csv("loan_data.csv")
```

```
# ----------------------------

# Handle Missing Values

# ----------------------------

df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])

df["Married"] = df["Married"].fillna(df["Married"].mode()[0])

df["Dependents"] = df["Dependents"].fillna(df["Dependents"].mode()[0])

df["Self_Employed"] = df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])


df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].median())

df["Loan_Amount_Term"] =
df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].mode()[0])

df["Credit_History"] = df["Credit_History"].fillna(df["Credit_History"].mode()[0])


# ----------------------------

# Convert Dependents

# ----------------------------

df["Dependents"] = df["Dependents"].replace("3+", "3")

df["Dependents"] = df["Dependents"].astype(int)


# ----------------------------

# Remove Loan_ID

# ----------------------------

df = df.drop("Loan_ID", axis=1)


# ----------------------------

# Convert Categorical Columns

# ----------------------------
```

```
df = pd.get_dummies(
    df,
    columns=[
        "Gender",
        "Married",
        "Education",
        "Self_Employed",
        "Property_Area"
    ],
    drop_first=True
)

# ------------------------------
# Convert Target Variable
# ------------------------------
df["Loan_Status"] = df["Loan_Status"].map({"Y": 1, "N": 0})

# ------------------------------
# Check Missing Values
# ------------------------------
print("\nMissing Values:\n")
print(df.isnull().sum())

# ------------------------------
# Features and Target
# ------------------------------
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

```
# ----------------------------
# Train-Test Split
# ----------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)

# ----------------------------
# Feature Scaling
# ----------------------------
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ----------------------------
# Logistic Regression Model
# ----------------------------
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

# ----------------------------
# Prediction
```

```
# ------------------------------
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# ------------------------------
# Accuracy
# ------------------------------
print("\nAccuracy :", accuracy_score(y_test, y_pred))

# ------------------------------
# Confusion Matrix
# ------------------------------
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:\n")
print(cm)

# ------------------------------
# Classification Report
# ------------------------------
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))

# ------------------------------
# ROC-AUC Score
# ------------------------------
auc = roc_auc_score(y_test, y_prob)
```

```
print("\nROC-AUC Score :", auc)

# ----------------------------
# Confusion Matrix Plot
# ----------------------------
plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

# ----------------------------
# ROC Curve
# ----------------------------
fpr, tpr, threshold = roc_curve(y_test, y_prob)

plt.figure(figsize=(6,5))

plt.plot(fpr, tpr, label="Logistic Regression")
plt.plot([0,1],[0,1],'r--')
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()

plt.show()
```